Abstract Factory para la persistencia de Los Atuendos

Aporte individual para integrar al documento de diseño del grupo

Patrones de Diseño de Software · Fundación Universitaria Compensar

1 de noviembre de 2026

Problema y elección del patrón

Los Atuendos necesita conservar prendas, personas, alquileres y operaciones de lavandería en una base relacional. Si cada gestor crea directamente sus DAO SQLite, la lógica de negocio queda vinculada a una implementación concreta. Se propone Abstract Factory para proporcionar una familia coherente de objetos de acceso a datos mediante interfaces (Gamma et al., 1994).

La fábrica abstracta DAOFactory declara siete métodos de obtención de DAO. SQLiteDAOFactory implementa el contrato y entrega los productos concretos que utilizan SQLiteConnection. Los gestores reciben las interfaces por constructor. La conexión y los DAO pertenecen a la misma configuración de archivo y esquema.

Tabla 1

Selección del patrón creacional

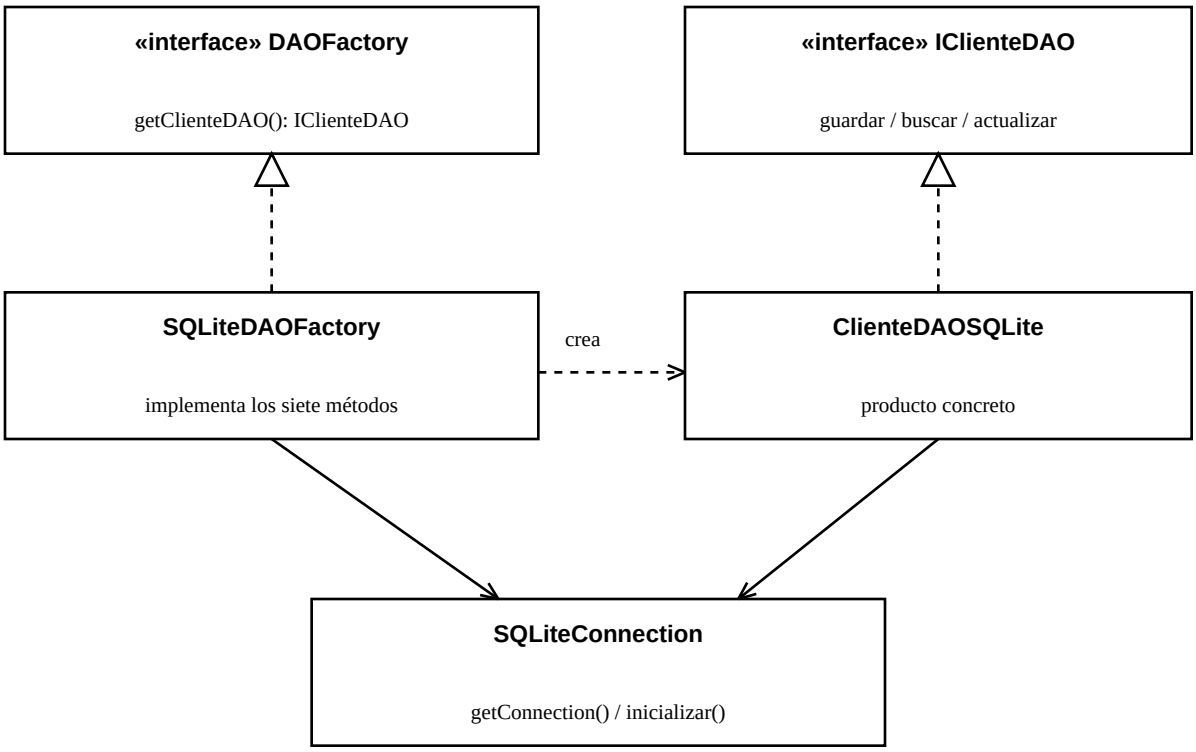
Patrón y tipo	Necesidad y aplicación	Justificación
Abstract Factory Creacional	Crear la familia de DAO de personas, clientes, empleados, prendas, alquileres, devoluciones y lavandería.	Centraliza la elección de tecnología y evita que cada gestor construya implementaciones SQLite.

Nota. Elaboración propia. Abstract Factory cubre la parte creacional de persistencia; el patrón estructural del grupo se documenta por separado.

Estructura y participantes

Figura 1

Diagrama de clases del patrón, vista representativa



Nota. Elaboración propia. Las flechas discontinuas verticales representan realización; la dependencia «crea» relaciona fábrica y producto. El archivo Mermaid adjunto amplía la figura a los siete pares de productos.

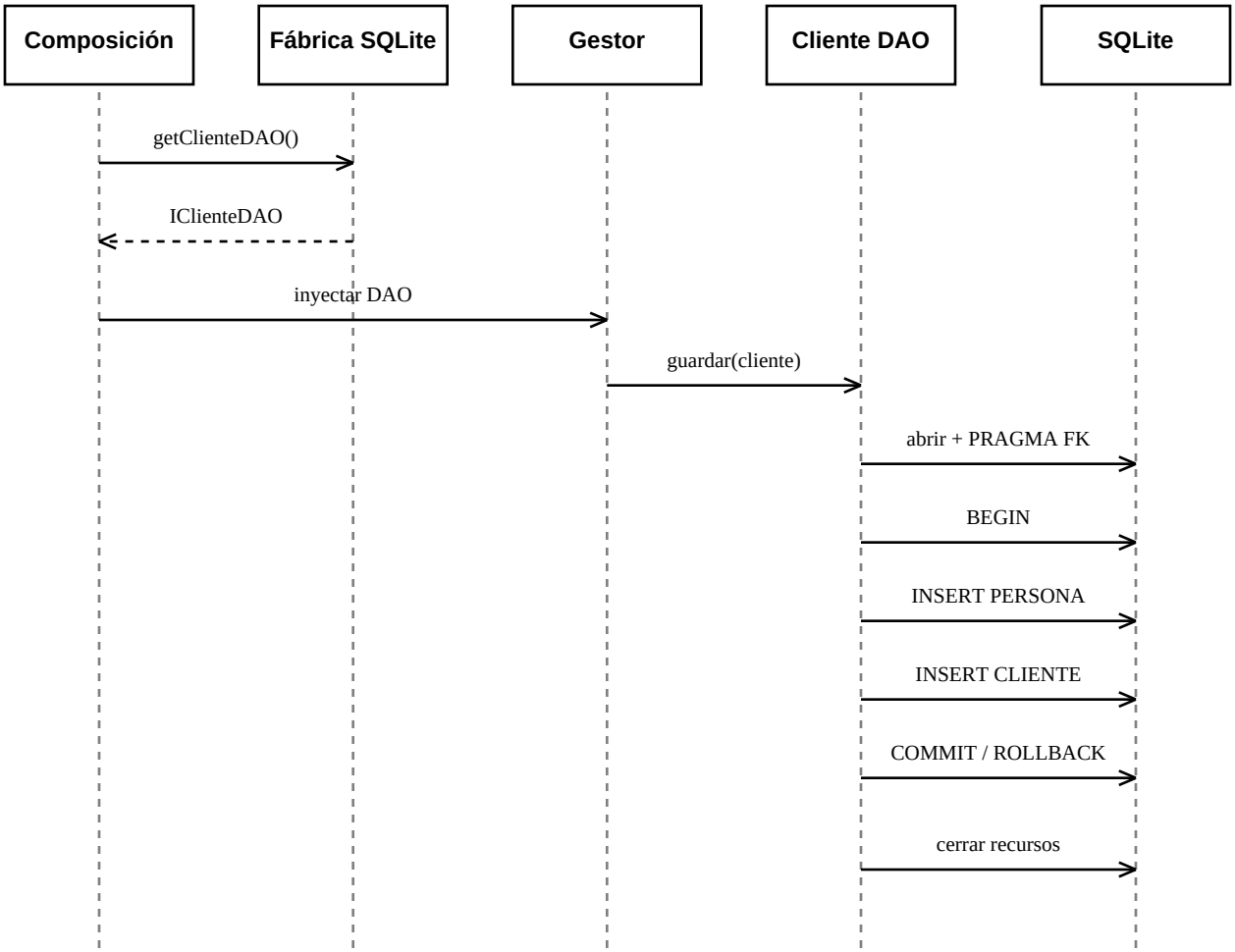
Los otros productos abstractos son IPersonaDAO, IEmpleadoDAO, IPrendaDAO, IAlquilerDAO, IDevolucionDAO e ILavanderiaDAO. Cada uno tiene una implementación homóloga terminada en DAOSQLite. DAOFactory corresponde al papel de fábrica abstracta y SQLiteDAOFactory al de fábrica concreta; los servicios son consumidores de los productos.

La fábrica no construye entidades de negocio como Cliente o VestidoDama. Esa responsabilidad continúa en Factory Method y Builder. SQLiteConnection abre conexiones independientes por operación; no se utiliza como conexión global compartida.

Interacción del patrón

Figura 2

Creación del producto y registro transaccional de un cliente



Nota. Elaboración propia. Vista resumida; el archivo de secuencia Mermaid explicita el gestor, el proveedor de conexión y las ramas de éxito y error.

El gestor invoca guardar sobre IClienteDAO. El producto concreto inserta PERSONA y CLIENTE con una única conexión y transacción. Si falla una operación, revierte los cambios y propaga PersistenciaException conservando la causa. El cliente de la fábrica no necesita conocer la clase concreta del producto.

Persistencia, integridad y verificación

La base contiene PERSONA y sus roles CLIENTE y EMPLEADO; PRENDA y tres especializaciones; ALQUILER, DETALLE_ALQUILER, DEVOLUCION, SOLICITUD_LAVANDERIA y LOTE_LAVANDERIA. Se aplican claves primarias, foráneas, unicidad, obligatoriedad y restricciones de dominio. Las claves foráneas se activan en cada conexión, de acuerdo con la documentación de SQLite (SQLite, s. f.-a).

Se utilizan sentencias preparadas para los parámetros. Los registros que afectan varias tablas se confirman o revierten como unidad (SQLite, s. f.-b). Al guardar un alquiler, el DAO adquiere el bloqueo de escritura antes de consultar reservas y consecutivo; verifica el intervalo inclusivo y persiste cabecera y detalles. El consecutivo es entero y se genera dentro de la transacción.

La consulta de pendientes no modifica la cola. Strategy selecciona las solicitudes según la política del grupo; el DAO confirma de manera atómica el lote, los estados de solicitudes y las prendas enviadas. La prioridad se guarda como enum y el identificador creciente resuelve empates de llegada en una misma fecha.

Se ejecutaron seis pruebas de integración con archivos SQLite temporales: creación de familia y reapertura; duplicados de personas y reversión; tres subtipos de prendas; consultas, consecutivos y conflicto de alquiler; referencias inexistentes y reversión de detalles; devolución y despacho atómico. Resultado: seis pruebas satisfactorias, cero fallos y cero errores. Esta evidencia no cubre la interfaz ni la concurrencia entre procesos.

El aporte conserva los contratos actuales del repositorio. No se afirma que toda la aplicación esté terminada: Main aún requiere ensamblar los gestores y la fachada. La guía de integración enumera las diferencias con los PDF de contexto y los ajustes de compilación. El grupo debe integrar esta sección, completar la portada y realizar pruebas de extremo a extremo.

Referencias

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

SQLite. (s. f.-a). *SQLite foreign key support*. https://www.sqlite.org/foreignkeys.html

SQLite. (s. f.-b). *Transaction*. https://www.sqlite.org/lang_transaction.html

Xerial. (s. f.). *SQLite JDBC driver* [Repositorio de código]. GitHub. https://github.com/xerial/sqlite-jdbc

Se consultaron también los documentos de arquitectura y contextualización suministrados por el equipo como material interno, sin atribuirles autores ni fechas no confirmados. El código se adapta al repositorio LosAtuendosLavanderia, revisión 337d24b481f02505857ebd26ebd807fdb179691f.